

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа

“ДЗ4: Технический дизайн микросервисной архитектуры”

Выполнил:

Преображенский Артемий

Группа:

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Разделить монолитное backend-приложение на микросервисы с соблюдением подхода database-per-service.
2. Спроектировать микросервисную архитектуру, описать взаимодействие между сервисами.
3. Разделить общую базу данных на несколько обособленных баз данных.
4. Спроектировать эндпоинты для межсервисного взаимодействия в формате OpenAPI.
5. Описать запросы, ответы и возможные ошибки.
6. Подготовить документ с требованиями и шагами по переходу от монолитного приложения к микросервисной архитектуре

Ход работы

1) Бекенд разделён на 3 независимых сервиса, каждый со своей БД и своей зоной ответственности.

- Auth Service (пользователи и аутентификация)

Публичные эндпоинты:

- POST /auth/register
- POST /auth/login
- GET /auth/me

- Catalog Service (каталог ресторанов)

Публичные эндпоинты:

- GET/POST /api/restaurants/
- GET/DELETE /api/restaurants/{id}/
- GET/POST /api/restaurants/{id}/menu
- GET/DELETE /api/restaurants/{id}/menu/{itemID}
- GET/POST /api/restaurants/{id}/tables
- GET/DELETE /api/restaurants/{id}/tables/{tableID}
- GET/POST /api/restaurants/{id}/reviews
- GET/PUT/DELETE /api/restaurants/{id}/reviews/{reviewID}

- Booking Service (бронирования)

Публичные эндпоинты (по заданию):

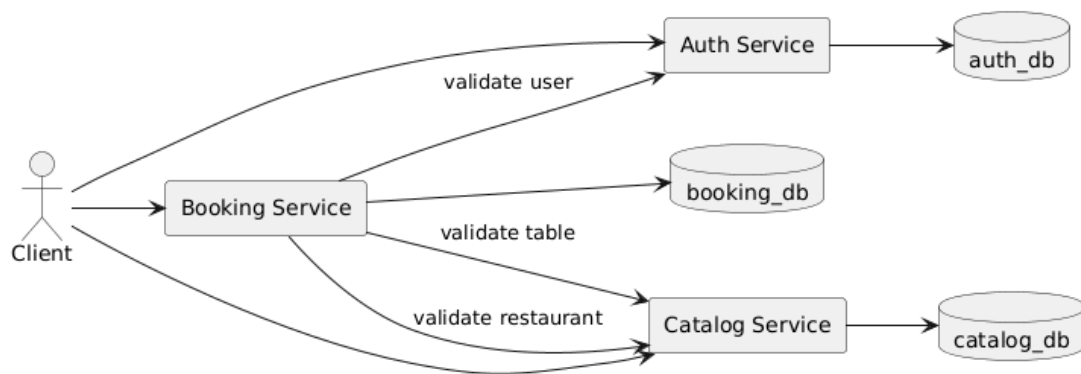
- GET /restaurants/{id}/tables/{tableID}/availability
- POST /bookings
- GET /me/bookings

- GET /me/bookings/{bookingID}
- (опционально в рамках задания) DELETE /me/bookings/{bookingID} или отдельный cancel-эндпоинт

2) Взаимосвязь микросервисов и способы взаимодействия

- Booking Service не читает таблицы пользователей/ресторанов напрямую. Для валидации и получения необходимых данных он обращается к внутренним техническим API других сервисов.
- Аутентификация выполняется через JWT: клиент передает токен в публичные эндпоинты, а сервисы извлекают user_id из токена для авторизации/аудита.

3) Разделение БД на обособленные БД (database-per-service)



4) Внутренние эндпоинты для межсервисного взаимодействия (OpenAPI)

Межсервисное HTTP API 1.0.0 OAS 3.0

auth-service

GET

/service/users/{id}

Пользователь по id (catalog-service → auth-service)

catalog-service

GET

/service/restaurants/{restaurantID}/tables/{tableID}

Стол по ресторану и id (booking-service → catalog-service)

5) Варианты запросов/ответов и возможные ошибки

1. Валидационные ошибки (400): неверный формат UUID в path-параметрах.
2. Не найдено (404): сущность отсутствует (user/restaurant/table).
3. Внутренняя ошибка (500): недоступность БД, ошибки инфраструктуры.
4. 401 Unauthorized: отсутствует/невалидный JWT.
5. 409 Conflict: конфликт при создании.

6) Пример корректной логики взаимодействия при создании бронирования

1. Клиент отправляет POST запрос в Booking Service (создание бронирования), передавая JWT.
2. Booking Service валидирует JWT и извлекает user_id.
3. Booking Service вызывает Auth Service: GET /service/users/{user_id}
 - если 404, возвращает клиенту ошибку (например, 404/401 — в зависимости от выбранной политики).
4. Booking Service вызывает Catalog Service: GET /service/restaurants/{restaurantID}/tables/{tableID}
 - если 404, возвращает клиенту ошибку «ресторан/столик не найден» (или «столик не принадлежит ресторану»).
5. Booking Service проверяет доступность по своей модели (например, пересечения по времени) и создаёт запись в booking_db.
6. Возвращает клиенту результат создания бронирования (и далее это бронирование доступно через /me/bookings и /me/bookings/{bookingID}).

Вывод

В ходе выполнения работы была спроектирована микросервисная архитектура для сервиса бронирования столиков в ресторанах. Система была разделена на три основных микросервиса: сервис аутентификации, сервис каталога ресторанов и сервис бронирований. Такое разделение лучше соответствует текущему API, уменьшает количество межсервисных зависимостей и позволяет сохранить архитектуру достаточно простой и понятной для дальнейшей реализации.